

SENG1050

Lab 10

In this lab, you will gain experience in encoding and compression through examples in Huffman encoding, CRC, and GZIP compression.

[Task 1*](#) - [Task 2*](#) - [Task 3](#)

Assessment 3 (Important!)

Work on Assessment 3 (Web-Based Assignment 2) and get your milestones graded by this week.

Task 1 - Huffman Encoding (100% in Exams)

As you have seen in the lecture, Huffman Encoding is a simple statistical compression technique. To construct a Huffman code:

1. Rank all symbols by frequency of occurrence.
2. Repeatedly combine the two symbols of the lowest probability to form a new composite symbol.
3. Then, build a binary tree where each node is labelled with the frequency of all nodes beneath it. Also, build the "trail" to each leaf - left is 0 right is 1.
4. Build the encoding table by following the trail for each letter. This trail is the Huffman Code for each letter.

For example, consider the following text:

the three men were meant

1. The symbols in order of frequency:

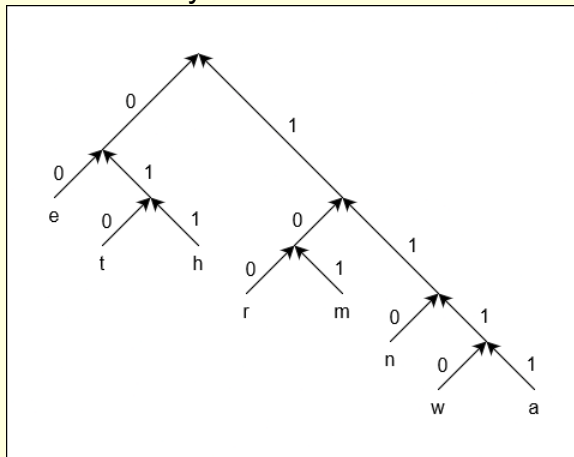
e	t	h	r	m	n	w	a
7	3	2	2	2	2	1	1

2. Now, repeatedly add the two smallest frequencies together. This combination is done from the leftmost to the rightmost column as follows:

e (7)	e (7)	e (7)	e (7)	e (7)	e (7)	{e{th}} (12)	{{e{th}}}{rm}{n{wa}} (20)
t (3)	t (3)	t (3)	t (3)	{th} (5)	{th} (5)		
h (2)	h (2)	h (2)	h (2)				
r (2)	r (2)	r (2)	{rm} (4)	{rm} (4)	{{rm}{n{wa}}} (8)	{{rm}{n{wa}}} (8)	
m (2)	m (2)	m (2)					
n (2)	n (2)	{n{wa}} (4)	{n{wa}} (4)	{n{wa}} (4)			

w (1)	{wa} (2)						
a (1)							

3. Draw a binary tree from this table. It should look as follows:



4. Building a table from this encoding tree gives the Huffman Encoding

e	t	h	r	m	n	w	a
00	010	011	100	101	110	1110	1111

5. At the end, the string would be encoded as:

0100110001001110000001010011011100010000101001111110010.

Ask your demonstrator for help if you have any problems.

As further practice, use Huffman Encoding to compress the phrase [antelopeavoidanteaters](#).

Assuming that it normally takes 8 bits to store a single ASCII character, how much has the message been compressed by the encoding? Calculate this compression rate as the proportion reduced using the formula: $1 - (\text{compressed_size} / \text{original_size})$.

Task 2 - CRC Calculation (100% in Exams)

To calculate the CRC of a message, we use the binary co-efficients of a generator polynomial as the divisor for the message we need to send.

- For example, given a message: [10011000](#), we want to calculate the CRC using the generator polynomial x^3+x+1 which can also be written as $1x^3+0x^2+1x^1+1x^0$. Taking the co-efficients of the polynomial gives us [1011](#).
- The CRC will always be the length of the highest power of x in the generator polynomial with a non-zero co-efficient. In this case, the highest power of x is 3, and so the CRC will be of length 3.
- We need to add trailing zeros to the end of the message that equals to the length of the CRC, resulting in [10011000`000](#), and perform the division.
- To perform the division, we will shift the generator co-efficients along the message until the left-most ones are lined up, then perform bitwise XOR. Continue this until all of the message bits are set to zero as follows:

```

10011000`000
1011
00101000`000
 1011
00000100`000
   101 1
00000001`100
    1 011
00000000`111

```

- At the end of the process, the bits in the remainder become the CRC, which we will append to the message to be sent. In this example, the transmitted message would be `10011000111`.
- The receiver can check the message by using the same generator to perform the division on the received message. The message is valid if the remainder is zero. The working is as follows:

```

10011000`111
1011
00101000`111
 1011
00000100`111
   101 1
00000001`011
    1 011
00000000`000

```

As further practice, calculate the CRC for the message `1010101101101001` using the generator `011011`. Verify your answer at the end.

Task 3 - GZIP Compression (Optional - Not in Exams)

GZIP compression is a form of substitutional compression that uses a window to keep track of the current output. (We are specifically using only the LZ77 algorithm component for this course). To perform GZIP compression:

- Perform a look-ahead in the input to find the longest pattern that exists in the window.
- If there is a match, output the optimal offset-length instruction and add the result to the window.
- Otherwise, output the first character and add it to the window. Repeat until reaching the end of the input.

For example, consider the string `ABBABAABABAB`.

INPUT	OUTPUT	WINDOW
A	A	A
B	B	AB
B	B	ABB
AB	(-3,2)	ABBAB
A	A	ABBABA
ABA	(-3,3)	ABBABAABA
BAB [End of message]	(-2,3)	ABBABAABABAB

The output would be `ABB(-3,2)A(-3,3)(-2,3)`.

As further practice, use GZIP compression to compress the following string: `ABCBACABACABCCBABBAAABCCBA`.